1. Implementation of Basic Array Operations (Insertion, Deletion, Traversal)

```
#define MAX 100

void display(const int a[], int n) {
   printf("Array elements: ");
   for (int i = 0; i < n; i++) printf("%d ", a[i]);
   printf("\n");
}

int main(void) {
   int a[MAX], n, choice, pos, value;
   printf("Enter number of elements: ");
   if (scanf("%d", &n) != 1 || n < 0 || n > MAX) { printf("Invalid size.\n"); return 1; }
   printf("Enter %d elements: ", n);
   for (int i = 0; i < n; i++) if (scanf("%d", &a[i]) != 1) return 1;
   do {
      printf("\n1. Insert\n2. Delete\n3. Traverse\n4. Exit\nEnter choice: ");
      if (scanf("%d", &choice) != 1) return 1;
      switch (choice) {
      case 1:
         if (n == MAX) { printf("Array is full. Cannot insert.\n"); break; }
         printf("Enter position (0 to %d) and value: ", n);
         if (scanf("%d %d", &pos, &value) != 2) return 1;
         if (pos < 0 || pos > n) { printf("Invalid position.\n"); break; }
         for (int i = n; i > pos; i--) a[i] = a[i - 1];
         a[pos] = value; n++; display(a, n); break;
```

```
        case 2:
            printf("Enter position to delete (0 to %d): ", n - 1);
            if (scanf("%d", &pos) != 1) return 1;
            if (pos < 0 || pos >= n) { printf("Invalid position.\n"); break; }
            for (int i = pos; i < n - 1; i++) a[i] = a[i + 1];
            n--; display(a, n); break;
        case 3: display(a, n); break;
        case 4: printf("Exiting...\n"); break;
        default: printf("Invalid choice.\n");
        }
    } while (choice != 4);
    return 0;
}
```

2. Building a Dynamic Employee Record System using Structure Pointers

```
#include <stdio.h>
#include <stdlib.h>

struct Employee { int id; char name[50]; char designation[30]; float salary; };

int main(void) {
    int n, searchId, found = 0;
    printf("Enter number of employees: ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > 1000) { printf("Invalid number of employees.\n");
return 1; }
    struct Employee *emp = malloc((size_t)n * sizeof *emp);
```

```
  if (emp == NULL) { printf("Memory allocation failed.\n"); return 1; }
  for (int i = 0; i < n; i++) {
    printf("\nEnter details for employee %d:\nID: ", i + 1);
    if (scanf("%d", &emp[i].id) != 1) { free(emp); return 1; }
    printf("Name: ");
    if (scanf("%49s", emp[i].name) != 1) { free(emp); return 1; }
    printf("Designation: ");
    if (scanf("%29s", emp[i].designation) != 1) { free(emp); return 1; }
    printf("Salary: ");
    if (scanf("%f", &emp[i].salary) != 1) { free(emp); return 1; }
  }
  printf("\n%-6s %-15s %-15s %-10s\n", "ID", "Name", "Designation", "Salary");
  for (int i = 0; i < n; i++) printf("%-6d %-15s %-15s %-10.2f\n", emp[i].id, emp[i].name,
emp[i].designation, emp[i].salary);
  printf("\nEnter employee ID to search: ");
  if (scanf("%d", &searchId) != 1) { free(emp); return 1; }
  for (int i = 0; i < n; i++) if (emp[i].id == searchId) {
    printf("Found: %d %s %s %.2f\n", emp[i].id, emp[i].name, emp[i].designation,
emp[i].salary); found = 1; break;
  }
  if (!found) printf("Employee not found.\n");
  free(emp);
  return 0;
}
```

3.

Implementation of Stack and Expression Conversion - Infix to Postfix

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 100
char stack[MAX]; int top = -1;
int precedence(char c) { if (c == '^') return 3; if (c == '*' || c == '/') return 2; if (c == '+' || c == '-') return 1; return 0; }
int rightAssociative(char c) { return c == '^'; }
int main(void) {
  char infix[MAX], postfix[MAX]; int j = 0, valid = 1;
  printf("Enter infix expression: ");
  if (!fgets(infix, sizeof infix, stdin)) return 1;
  infix[strcspn(infix, "\n")] = '\0';
  for (int i = 0; infix[i] != '\0'; i++) {
    char c = infix[i];
    if (isspace((unsigned char)c)) continue;
    if (isalnum((unsigned char)c)) postfix[j++] = c;
    else if (c == '(') { if (top == MAX - 1) { valid = 0; break; } stack[++top] = c; }
    else if (c == ')') {
      while (top >= 0 && stack[top] != '(') postfix[j++] = stack[top--];
      if (top < 0) { valid = 0; break; } top--;
    } else if (precedence(c)) {
      while (top >= 0 && stack[top] != '(' &&
          (precedence(stack[top]) > precedence(c) ||
          (precedence(stack[top]) == precedence(c) && !rightAssociative(c))))
postfix[j++] = stack[top--];
      if (top == MAX - 1) { valid = 0; break; } stack[++top] = c;
    } else { valid = 0; break; }
    if (j >= MAX - 1) { valid = 0; break; }
```

```
  }
  while (valid && top >= 0) { if (stack[top] == '(') { valid = 0; break; } postfix[j++] =
stack[top--]; }
  postfix[j] = '\0';
  if (!valid) { printf("Invalid expression.\n"); return 1; }
  printf("Postfix expression: %s\n", postfix); return 0;
}
```

4. Postfix Expression Evaluation using Stack

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <math.h>
#define MAX 100
int main(void) {
  char expr[MAX]; double stack[MAX]; int top = -1;
  printf("Enter postfix expression (single-digit operands, e.g. 23+4*): ");
  if (!fgets(expr, sizeof expr, stdin)) return 1;
  expr[strcspn(expr, "\n")] = '\0';
  for (int i = 0; expr[i] != '\0'; i++) {
    char c = expr[i]; if (isspace((unsigned char)c)) continue;
    if (isdigit((unsigned char)c)) { if (top == MAX - 1) { printf("Stack overflow.\n"); return 1;
} stack[++top] = c - '0'; }
    else if (c == '+' || c == '-' || c == '*' || c == '/' || c == '^') {
      if (top < 1) { printf("Invalid postfix expression.\n"); return 1; }
      double op2 = stack[top--], op1 = stack[top--], result;
      switch (c) {
      case '+': result = op1 + op2; break; case '-': result = op1 - op2; break;
```

```
        case '*': result = op1 * op2; break;
        case '/': if (op2 == 0) { printf("Division by zero.\n"); return 1; } result = op1 / op2;
break;
        default: result = pow(op1, op2);
        }
        stack[++top] = result;
      } else { printf("Invalid character: %c\n", c); return 1; }
   }
   if (top != 0) { printf("Invalid postfix expression.\n"); return 1; }
   printf("Result = %.2f\n", stack[top]); return 0;
}
```

5. Implementation of Simple Queue using Array

```
#include <stdio.h>
#define MAX 5
int queue[MAX], front = -1, rear = -1;
void enqueue(int value) {
   if (rear == MAX - 1) { printf("Queue overflow\n"); return; }
   if (front == -1) front = 0;
   queue[++rear] = value; printf("%d enqueued.\n", value);
}
void dequeue(void) {
   if (front == -1) { printf("Queue underflow\n"); return; }
   printf("%d dequeued.\n", queue[front++]);
   if (front > rear) front = rear = -1;
}
```

```
void display(void) {
   if (front == -1) { printf("Queue is empty.\n"); return; }
   printf("Queue elements: "); for (int i = front; i <= rear; i++) printf("%d ", queue[i]); printf("\n");
}
int main(void) {
   int choice, value;
   do {
      printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\nEnter choice: ");
      if (scanf("%d", &choice) != 1) return 1;
      switch (choice) { case 1: printf("Enter value to enqueue: "); if (scanf("%d", &value) != 1) return 1; enqueue(value); break;
      case 2: dequeue(); break; case 3: display(); break; case 4: printf("Exiting...\n"); break; default: printf("Invalid choice.\n"); }
   } while (choice != 4); return 0;
}
```

6. Implementation of Priority Queue using Unordered Array

```
#include <stdio.h>
#define MAX 100
struct Element { int data, priority; } pqueue[MAX]; int n = 0;
void insert(int data, int priority) {
   if (n == MAX) { printf("Priority queue is full.\n"); return; }
   pqueue[n++] = (struct Element){data, priority}; printf("Inserted %d with priority %d.\n", data, priority);
}
void deleteHighestPriority(void) {
```

```
    if (n == 0) { printf("Priority queue is empty.\n"); return; }
    int idx = 0; for (int i = 1; i < n; i++) if (pqueue[i].priority < pqueue[idx].priority) idx = i;
    printf("Removed %d (priority %d).\n", pqueue[idx].data, pqueue[idx].priority);
    for (int i = idx; i < n - 1; i++) pqueue[i] = pqueue[i + 1];
    n--;
}
void display(void) {
    if (n == 0) { printf("Priority queue is empty.\n"); return; }
    printf("Data\tPriority\n"); for (int i = 0; i < n; i++) printf("%d\t%d\n", pqueue[i].data,
pqueue[i].priority);
}
int main(void) {
    int choice, data, priority;
    do {
        printf("\n1. Insert\n2. Delete Highest Priority\n3. Display\n4. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) return 1;
        switch (choice) { case 1: printf("Enter data and priority: "); if (scanf("%d %d", &data,
&priority) != 2) return 1; insert(data, priority); break;
        case 2: deleteHighestPriority(); break; case 3: display(); break; case 4:
printf("Exiting...\n"); break; default: printf("Invalid choice.\n"); }
    } while (choice != 4); return 0;
}
```

7. Implementation of Singly Linked List (Insertion, Deletion, and Traversal)

```
#include <stdio.h>

#include <stdlib.h>
```

```
struct Node { int data; struct Node *next; }; struct Node *head = NULL;

void insertAtEnd(int value) {

    struct Node *node = malloc(sizeof *node); if (!node) { printf("Memory

allocation failed.\n"); return; }

    node->data = value; node->next = NULL;

    if (!head) { head = node; return; } struct Node *temp = head; while (temp-

>next) temp = temp->next; temp->next = node;

}

void insertAtBeginning(int value) {

    struct Node *node = malloc(sizeof *node); if (!node) { printf("Memory

allocation failed.\n"); return; }

    node->data = value; node->next = head; head = node;

}

void deleteValue(int value) {

    struct Node *temp = head, *prev = NULL;
```

```
    while (temp && temp->data != value) { prev = temp; temp = temp->next; }
    if (!temp) { printf("Value %d not found.\n", value); return; }
    if (!prev) head = temp->next; else prev->next = temp->next;
    free(temp); printf("Deleted %d.\n", value);
}
void traverse(void) {
    if (!head) { printf("List is empty.\n"); return; }
    printf("List: "); for (struct Node *t = head; t; t = t->next) printf("%d -> ", t-
>data); printf("NULL\n");
}
void cleanup(void) { while (head) { struct Node *t = head; head = head->next;
free(t); } }
int main(void) {
    int choice, value;
    do {
```

```
        printf("\n1. Insert at End\n2. Insert at Beginning\n3. Delete\n4.
Traverse\n5. Exit\nEnter choice: ");

        if (scanf("%d", &choice) != 1) { cleanup(); return 1; }

        switch (choice) { case 1: printf("Enter value: "); if (scanf("%d", &value) !=
1) { cleanup(); return 1; } insertAtEnd(value); break;

        case 2: printf("Enter value: "); if (scanf("%d", &value) != 1) { cleanup();
return 1; } insertAtBeginning(value); break;

        case 3: printf("Enter value to delete: "); if (scanf("%d", &value) != 1) {
cleanup(); return 1; } deleteValue(value); break;

        case 4: traverse(); break; case 5: printf("Exiting...\n"); break; default:
printf("Invalid choice.\n"); }

    } while (choice != 5); cleanup(); return 0;

}
```

8. Implementation of Circular Queue using Linked List

```
#include <stdio.h>

#include <stdlib.h>

struct Node { int data; struct Node *next; }; struct Node *front = NULL, *rear =

NULL;

void enqueue(int value) {

    struct Node *node = malloc(sizeof *node); if (!node) { printf("Memory

allocation failed.\n"); return; }

    node->data = value;

    if (!front) { front = rear = node; node->next = node; }

    else { node->next = front; rear->next = node; rear = node; }

    printf("%d enqueued.\n", value);

}

void dequeue(void) {

    if (!front) { printf("Queue underflow\n"); return; }

    struct Node *temp = front; printf("%d dequeued.\n", temp->data);
```

```
    if (front == rear) front = rear = NULL;
    else { front = front->next; rear->next = front; }
    free(temp);
}
void display(void) {
    if (!front) { printf("Queue is empty.\n"); return; }
    struct Node *temp = front; printf("Queue elements: ");
    do { printf("%d ", temp->data); temp = temp->next; } while (temp != front);
printf("\n");
}
int main(void) {
    int choice, value;
    do {
        printf("\n1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) return 1;
```

```
        switch (choice) { case 1: printf("Enter value: "); if (scanf("%d", &value) !=
1) return 1; enqueue(value); break;
        case 2: dequeue(); break; case 3: display(); break; case 4:
printf("Exiting...\n"); break; default: printf("Invalid choice.\n"); }
    } while (choice != 4);
    while (front) { struct Node *t = front; if (front == rear) front = rear = NULL;
else { front = front->next; rear->next = front; } free(t); }
    return 0;
}
```

9. Implementation and Traversal (In-order, Pre-order, Post-order) of a Binary Search Tree

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
```

```
struct Node *createNode(int value) { struct Node *node = malloc(sizeof *node);
if (!node) { fprintf(stderr, "Memory allocation failed.\n"); exit(1); } node->data
= value; node->left = node->right = NULL; return node; }

struct Node *insert(struct Node *root, int value) { if (!root) return
createNode(value); if (value < root->data) root->left = insert(root->left, value);
else if (value > root->data) root->right = insert(root->right, value); return root; }

void inorder(struct Node *r) { if (r) { inorder(r->left); printf("%d ", r->data);
inorder(r->right); } }

void preorder(struct Node *r) { if (r) { printf("%d ", r->data); preorder(r->left);
preorder(r->right); } }

void postorder(struct Node *r) { if (r) { postorder(r->left); postorder(r->right);
printf("%d ", r->data); } }

void freeTree(struct Node *r) { if (r) { freeTree(r->left); freeTree(r->right);
free(r); } }

int main(void) {
    struct Node *root = NULL; int n, value;
```

```
    printf("Enter number of nodes: "); if (scanf("%d", &n) != 1 || n < 0 || n >
10000) { printf("Invalid number of nodes.\n"); return 1; }
    printf("Enter %d values: ", n); for (int i = 0; i < n; i++) { if (scanf("%d",
&value) != 1) { freeTree(root); return 1; } root = insert(root, value); }
    printf("\nIn-order traversal: "); inorder(root); printf("\nPre-order traversal: ");
preorder(root); printf("\nPost-order traversal: "); postorder(root); printf("\n");
freeTree(root); return 0;
}
```

10. Implement Graph Search Algorithms (BFS & DFS)

```
#include <stdio.h>
#define MAX 20
int adj[MAX][MAX], visited[MAX], n;
void bfs(int start) {
```

```
    int queue[MAX], front = 0, rear = 0, seen[MAX] = {0}; seen[start] = 1;
queue[rear++] = start;
    printf("BFS traversal: ");
    while (front < rear) { int v = queue[front++]; printf("%d ", v); for (int i = 0; i
< n; i++) if (adj[v][i] && !seen[i]) { seen[i] = 1; queue[rear++] = i; } }
    printf("\n");
}
void dfsVisit(int v) { visited[v] = 1; printf("%d ", v); for (int i = 0; i < n; i++) if
(adj[v][i] && !visited[i]) dfsVisit(i); }
void dfs(int start) { for (int i = 0; i < n; i++) visited[i] = 0; printf("DFS traversal:
"); dfsVisit(start); printf("\n"); }
int main(void) {
    int edges, u, v, start;
    printf("Enter number of vertices: "); if (scanf("%d", &n) != 1 || n < 1 || n >
MAX) { printf("Invalid number of vertices.\n"); return 1; }
```

```
    printf("Enter number of edges: "); if (scanf("%d", &edges) != 1 || edges < 0 ||
edges > n * (n - 1) / 2) { printf("Invalid number of edges.\n"); return 1; }
    printf("Enter edges (u v), vertices numbered 0 to %d:\n", n - 1);
    for (int i = 0; i < edges; i++) { if (scanf("%d %d", &u, &v) != 2 || u < 0 || u >=
n || v < 0 || v >= n) { printf("Invalid edge.\n"); return 1; } adj[u][v] = adj[v][u] =
1; }
    printf("Enter starting vertex: "); if (scanf("%d", &start) != 1 || start < 0 || start
>= n) { printf("Invalid starting vertex.\n"); return 1; }
    bfs(start); dfs(start); return 0;
}
```

11. Implementation of Bubble, Selection and Heap Sorts

Data Structures and Algorithms Lab — Experiments 1–10

C programs based on the first 10 experiments in the provided DSA Lab Manual.

Experiments

1. Array operations (insertion, deletion, traversal)

2. Dynamic employee records using structures and pointers

3. Infix-to-postfix conversion using a stack

4. Postfix expression evaluation using a stack

5. Simple queue using an array

6. Priority queue using an unordered array (lower number = higher priority)

7. Singly linked list (insert, delete, traverse)

8. Circular queue using a linked list

9. Binary search tree traversals (in-order, pre-order, post-order)

10. Graph traversal using BFS and DFS

Compile and run

Use GCC from a terminal in this folder:

```sh

gcc -std=c11 -Wall -Wextra -pedantic 01_array_operations.c -o exp01

./exp01

```

For Experiment 4, link the math library on Linux/macOS GCC:

`gcc -std=c11 -Wall -Wextra -pedantic 04_postfix_evaluation.c -o exp04 -lm`

Each program is standalone. Input is read from the terminal. These implementations include basic input validation and memory-allocation checks.

12. Implementation of Hashing with Collision Resolution using Linear Probing

Compile and Sample-Test Results

All 10 C programs compiled with GCC using `-std=c11 -Wall -Wextra -pedantic`. Experiment 4 was linked with `-lm` for `pow()`.

Experiment	Sample test	Verified result
1. Array operations	Insert 25 at index 2, delete index 1	`10 20 25 30 40 50`, then `10 25 30 40 50`
2. Employee records	Two records; search ID 101	`Found: 101 Arun Manager 55000.00`
3. Infix to postfix	`(A+B)*(C-D)`	`AB+CD-*`
4. Postfix evaluation	`23+4*`	`Result = 20.00`
5. Simple queue	Enqueue 10, 20; display; dequeue; display	`10 20`, then `20` remains
6. Priority queue	Insert `(10,3)`, `(20,1)`, `(30,2)`; delete	Removes `20` with priority `1`; remaining entries are 10 and 30

| 7. Singly linked list | Append 10, 20; prepend 5; delete 10 | `5 -> 10 -> 20 -> NULL`, then `5 -> 20 -> NULL` |

| 8. Circular queue | Enqueue 10, 20, 30; dequeue once | `10 20 30`, then `20 30` |

| 9. BST traversals | Insert `50 30 70 20 40 60 80` | In-order `20 30 40 50 60 70 80`; pre-order `50 30 20 40 70 60 80`; post-order `20 40 30 60 80 70 50` |

| 10. BFS and DFS | Graph edges `0-1, 0-2, 1-3, 2-4, 3-4`, start 0 | BFS `0 1 2 3 4`; DFS `0 1 3 4 2` |

****Status: 10/10 sample tests passed.****

Notes:

- Experiment 2 accepts single-word names and designations, matching the lab manual's `%s`-style input.

- Experiment 4 accepts single-digit operands, as stated in the lab manual.

- Graph vertices are numbered `0` through `n-1`; the graph is undirected,

matching the manual.